CSE348 DATABASE MANAGEMENT SYSTEMS

SPRING 2026

TERM PROJECT

DUE: 30.05.2026 23:59

DEMO SELECTION DEADLINE: 31.05.2026 23:59

MINITUBE — A YOUTUBE CLONE

In this project, you need to develop a simplified video-sharing web application inspired by YouTube. To accomplish this, you will need a database to store users, channels, videos, subscriptions and comments; an HTML file to provide access to the application through a browser; and PHP code to handle interactions between the browser and the database.

Each required file is explained below:

- **index.html:** The initial page of the application. It includes a button to initialize the database. After clicking this button, the database needs to be created and the login page should be displayed to the user.
- **install.php:** Contains setup-related code for the database, such as creating the database, creating tables, filling these tables, etc. After this file is executed, the database needs to be ready.
- **generate_data.php:** Used to generate datasets for the specified entities.
- **login.html, login.php:** Handles user authentication. If the user is authenticated, the user needs to be redirected to the homepage.
- **feed.html, feed.php:** Displays the homepage of the user, which shows the latest videos from channels the user follows and a list of the most popular channels.
- **channel.html, channel.php:** Shows channel details, including channel information, the channel's videos and a subscribe / unsubscribe button.
- **watch.html, watch.php:** Displays a single video, its information and its comments. Adds a new comment and updates the view count.
- **sql.html, sql.php:** A general page that takes an SQL query from the user, runs it and shows the result.

PART I: CREATING DATABASE AND TABLES OF THE PROJECT

Prepare an installation page with PHP (the outline of it is given in `install.php` file) and when clicked on the installation button via HTML page, it should create the tables described below and fill these tables with the information described in Part II. **Do not change given table names (they are written in bold).**

(i) Create a database which is named by your name (e.g. `name_surname`).

(ii) Create a table for keeping **USERS**.

(iii) Create a table for **CHANNELS**.

(iv) Create a table for **VIDEOS**.

(v) Create a table for **SUBSCRIPTIONS**.

(vi) Create a table for **COMMENTS**.

PART II: INSERTING ELEMENTS TO THE TABLES

The required attributes for each table are listed below. If necessary, you may add additional attributes but given ones **must** be on your table. You must insert the minimum required data before the system can function properly. **Do not change the given attribute and table names.**

- **USERS** (minimum 100 users): user_id, username, password, user_image (an url), full_name, email, country, joined_on (needs to be date), bio. The `user_image` field may hold real random image links so they can be opened.
- **CHANNELS** (minimum 50 channels): channel_id, owner_id, channel_image (an url), name, description, created_on (a date), category. The `channel_image` field may hold real random image links so they can be opened.
- **VIDEOS** (minimum 200 videos): video_id, channel_id, title, description, url, duration_seconds, uploaded_at, view_count, like_count. The `url` field may hold real YouTube video links so they can be opened from the watch page.
- **SUBSCRIPTIONS** (minimum 120 subscriptions): subscription_id, subscriber_id, channel_id, subscribed_at
- **COMMENTS** (minimum 150 comments, at least 20 of which are replies): comment_id, video_id, user_id, parent_comment_id, body, posted_at

A user may own at most one channel. A subscription is the relationship between a user and a channel. A comment may be a reply to another comment, in which case it references its parent through `parent_comment_id` (replies have a non-NULL value; top-level comments have NULL).

Write a `generate_data.php` script that generates data for each entity. This php code will read data from the text file and by using given values, it will create different combinations. The size of the input file needs to be given wisely. For instance, if we create 30 users, at least we need 25 first names. This php code creates a `.sql` file that includes INSERT INTO statements. The input/output example is given below:

```
// first_names.txt – read this file as an input line by line
Ahmet
Mehmet
```

```
Ayşe
Fatma
Mustafa
Ali
…

// seed.sql – after creating this file, read this to insert those statements
into the created tables
INSERT INTO users VALUES (1, 'ahmet01', ...);
INSERT INTO users VALUES (2, 'mehmet02', ...);
INSERT INTO users VALUES (3, 'ayse03', ...);
INSERT INTO users VALUES (4, 'fatma04', ...);
…
```

PART III: DESIGNING THE WEBSITE

GENERAL RULES

- You are free to design your pages creatively. Well-designed pages and additional useful features may earn some bonus points; however, you are expected to come up with ideas on your own.
- When designing the application, make sure to guide the user properly. For example, if certain conditions are not met, display appropriate error messages.
- In addition, any operation that results in an update to the data must be handled correctly.

INITIAL PAGE

When the system starts, this page should display only a single button. When this button is clicked, it should initialize the database (creates tables, fills them and so on). The page title must be your name. After initialization, the login page should be displayed.

LOGIN PAGE

The system must have a login page where users can authenticate with their username and password. After authentication, each user must be welcomed by his or her name “Hello, Name!”, which is the title of the page.

After a successful login, redirect the user to the homepage with the user identifier carried as a URL parameter (e.g. `feed.php?user_id=5`). Every subsequent page reads the user from `$_GET` and forwards it in its own links and forms.

AFTER AUTHENTICATION

After successful user authentication, the page should display the latest videos from the channels the user is subscribed to on the left side, ordered by upload date. Each row should include the video title, the channel name, the uploader’s country and how many days ago the video was uploaded.

On the right side, the upper section should display the five channels with the highest number of subscribers across the whole platform, along with their subscriber counts. The lower section should show the user's profile (full name, country, join date, …).

Each video title should be a link that opens the video page (`watch.php`). Each channel name and image should be a link that opens the channel page (`channel.php`). All internal links must forward the current `user_id`.

CHANNEL PAGE

All information about the selected channel should be displayed: channel name, channel_image, category, owner's full name, owner's country, creation date and the current number of subscribers. The description of the channel must be handled when it is missing — display "(no description)" instead of an empty cell.

Below the channel information, list every video uploaded by this channel, with the title, duration, upload date and current view count. Each title is a link to the video page.

At the top of the page, include a Subscribe / Unsubscribe button. If the user is not yet subscribed to this channel, the button text is "Subscribe" and clicking it adds a new row to the subscription table. If the user is already subscribed, the button text is "Unsubscribe" and clicking it removes the existing row. After the action, the page should be re-rendered so that the subscriber count and the button text reflect the new state.

VIDEO PAGE

This page displays a single video together with its information. The page must show the video title, channel link, uploader country, formatted duration (e.g. `3:42`), upload date, total view count and a popularity badge. The badge is computed from the view count: "Popular" for at least 1000 views, "Trending" for at least 100, and "New" otherwise. The badge text must be produced by an SQL `CASE/IF` expression — do not compute it in PHP.

Every time the page loads, the view count of the video must be incremented by one.

Below the video, display the comment thread. Top-level comments come first, in reverse chronological order. Each top-level comment must show the replies to it indented underneath it. The thread must be retrieved with a single SQL query that performs a self-join on the comment table.

At the bottom of the page, include a form that allows the current user to post a new (top-level) comment. After submission, the page should be reloaded so the new comment becomes visible.

ADDITIONAL GENERAL OPERATIONS

Create an additional page for arbitrary SQL queries. The page contains a text area in which the user types an SQL query and a button to execute it. The result of the query should be displayed on the same page:

- If the query is a `SELECT`, render the result as an HTML table whose column headers come from the result-set columns. You can set a limit as 10 rows.
- If the query is an `INSERT`, `UPDATE` or `DELETE`, display affected rows.
- If the query is invalid, display the error message.

Above the result, echo the executed query in a block so that what was run is visible.

EXAMPLE PAGE DESIGN LAYOUTS

This part is up to you — design it on your own. Use additional wrapper / container tags such as `<div></div>` to properly structure and divide the page.

DEMO AND SUBMISSION RULES

- You should design the database with the given entities and attributes.
- Create the **ER diagram** and the **relational diagram** of the database. You should create your tables according to the diagrams you draw, and justify your design choices.
- Create a visual representation that illustrates the **action flow** of your design.
- The demo days will be announced on Yulearn on **`31.05.2026`**. You need to select your demo slot until **`31.05.2026`** **23:59**. You must attend the demo **on time**; if you do not attend, even if you submitted a project, your grade will be 0.
- Submit your ER diagram (`FullName_StudentID_ER.pdf`), relational diagram (`FullName_StudentID_Relational.pdf`), action flow (`FullName_StudentID_ActionFlow.pdf`), SQL, PHP and HTML files → (`FullName_StudentID.zip`).
- Add comments that include your name and brief explanations in each file that you created.
- During the demo, you will be asked to write a short SQL query live on your own and to walk through any part of your code.
- Submit your own work. Plagiarism will not be tolerated.